

C23 — Machine Learning, Ranking and Recommendation for PMs

1. What interviewers test

"Improve the recommendations in a music app for a user who plays the same ten songs."
"Search results for a class of queries got worse. How do you find out why?" "A new ranking raises time spent but users say they feel worse. What do you do?" "Choose the threshold for a fraud classifier." These questions draw on the classical machine-learning ideas behind feeds, search, recommendations and risk scores. A PM is not asked to train a model. A PM is asked to define the decision a model supports, understand what the model learns from, choose and read the right metrics, see how the system can go wrong (including through its own outputs), and design experiments that can tell whether a change helped.

Where it shows up. Inferred from the round structures candidates commonly report: ranking, recommendation, search and classification questions at Google, Meta, Microsoft, Amazon, Apple, Netflix, Stripe and other consumer-scale and payments companies, in product, analytical and AI-focused rounds [unverified: candidate-reported]. Questions on large language models are in C24 and C25; this chapter covers the older, still dominant ideas behind them. On the job it is the work of PMs on feeds, search, recommendations, fraud and trust-and-safety systems.

What is scored. Whether you treat a model as a means to a product outcome Inferred. Interviewers listen for a clear decision and label; error costs that set thresholds; an understanding of stages in a ranking system; metrics that separate engagement from satisfaction and short from long term; awareness of data problems (bias, noise, cold start) and of feedback loops; and a sound experiment plan that accounts for novelty and interference.

Six observable behaviors:

1. You state what the model predicts, from what data, and what decision it supports.
2. You choose thresholds and operating points from the costs of each error type and the base rate.
3. You describe a ranking system in stages and say where you would intervene.
4. You choose a metric stack that separates engagement from satisfaction and short from long term, with guardrails.
5. You name data problems and feedback loops, and design for exploration.
6. You plan experiments on ranking systems with their pitfalls: novelty, interference and slow effects.

The failure this prevents. Six patterns end ranking answers: "use machine learning" with no decision or label, accuracy quoted for a rare event, a single click metric, ignoring cold start and feedback loops, an offline improvement declared a product win, and no view of what the system does to the people and creators who feed it.

2. The concept from first principles

C23.1 Why ML literacy matters for PMs, and what interviewers test

Many product surfaces are, underneath, predictions: which items to show, which messages are spam, which payments are fraud, how long a delivery will take. A PM who understands the shape of those predictions can set the right goal, ask engineers the right questions, interpret results correctly and notice when a metric improved for the wrong reason. A PM who does not will accept "the model is better" without asking better at what, measured how, on whom, and at what cost.

What this chapter is not. It does not teach how to build models. It teaches the product-level concepts: what a prediction needs, how to judge it, how ranking systems are staged, how they fail, and how to test changes. The next chapters build on it for language models (C24) and for building and evaluating AI products (C25).

Counterexample. Asked how to improve a feed, a candidate says: "Use a deep learning model trained on engagement to rank posts." The interviewer asks what the model should predict, for whom, and whether more engagement is the goal. The candidate has treated "machine learning" as the answer and has not said what decision it serves or what could go wrong when the system optimizes clicks. A better opening: "First, what is the feed for? If it is to help people see things they value, clicks are only a rough proxy, so I'd define what we want the model to predict and how we'd check that it matches what people value."

Hidden risks. Two. *Proxy worship*: optimizing what is easy to measure (clicks) can diverge from what matters (satisfaction), as in the Netflix Prize case (CS-C23-02). *Treating the model as the product*: data, thresholds, interface and incentives often matter more than the algorithm.

C23.2 Prediction in plain terms: features, labels, training, inference, evaluation

A **model** is a function that has been fitted to data to predict something about new cases. Its parts (FW-C23-01):

- **Training data** is the set of past examples a model learns from.
- A **label** is the outcome the model is trying to predict for each training example, such as "this email was marked as spam," "this video was watched to the end," or "this payment was later disputed as fraud."

- *Features* are the inputs the model uses to predict: attributes of the user, the item and the context (time of day, device, past behavior).
- *Training* is fitting the model to the examples; *inference* is using the trained model on a new case to produce a prediction (a score).
- *Evaluation* checks how well the model predicts on examples it did not train on (held-out data), which estimates how it will do on the future.

Overfitting. **Overfitting** is a model learning the quirks of its training data so closely that it predicts new cases poorly. A model that scores 99% on the data it trained on and 80% on held-out data has overfitted. The PM-level consequence: always ask for performance on data the model has not seen, and on data that looks like the future (recent, and from the segments that matter).

The PM's questions. What decision does the prediction support, and who acts on it? What is the label, and does it really measure what we care about? (A label such as "clicked" measures interest in a headline, not value from the article.) What data do we have, from whom, and what is missing? What does each error cost? How will we know the model is working after launch, and what happens when the world changes (a model trained on last year's behavior may decay)? This is the prediction brief.

When not to use a model. If a simple rule does the job and is easier to explain, audit and change, use it: "hold a payment above \$10,000 from a new account" is a rule; a model earns its place when patterns are too many and subtle for rules. A model also needs enough labeled data and a way to learn whether it is right.

C23.3 Classification and thresholds: precision, recall, and the cost of each error type

Many decisions are yes or no: is this message spam, is this payment fraud, is this post harmful? A *classifier* produces a score, and a **threshold** is the score above which the system acts. Moving the threshold trades one kind of error for another.

Errors. A *false positive* is flagging something that is fine; a *false negative* is missing something that is bad. **Precision and recall** are two measures of a classifier: precision is the share of flagged items that are correct, and recall is the share of all true items that were flagged; raising one often lowers the other. Precision answers "when we flag, how often are we right?" Recall answers "of all the bad things, how many did we catch?"

Base rates. When the thing you are looking for is rare, even an accurate classifier flags mostly good items (C12). Take 1,000,000 items a day with 1% bad: 10,000 bad and 990,000 good. A classifier with 80% recall and a 0.8% false-positive rate catches 8,000 and wrongly flags about 7,900: a precision of about 50%. If only 0.1% are bad (1,000 bad), the same classifier catches 800 and wrongly flags about 8,000: a precision of about 9%. The classifier did not change; the base rate did. Accuracy is misleading for rare events: a model that never flags anything is 99% accurate when 1% are bad, and catches nothing.

Choosing the threshold by cost (FW-C23-02). Assign a cost to each error (a wrongly blocked good item costs a review and a frustrated user; a missed bad item costs harm and cleanup), compute total cost at several thresholds, and add practical limits (how many flags humans can review in a day). Worked example 1 does this with numbers. The right threshold is not the one with the highest accuracy; it is the one that minimizes the cost you care about within your constraints. Often the best design has more than two outcomes: act automatically on very high scores, send a middle band to human review, and allow the rest (Stripe-style fraud design, IQ-C23-09).

Calibration. A score is *calibrated* if items scored 0.8 are really bad about 80% of the time. Calibrated scores let you set thresholds by cost and combine them with other information; uncalibrated scores only rank. Ask whether scores are calibrated if you plan to use them as probabilities.

Why it works. Cost-based thresholds make the tradeoff explicit and defensible. **When not to optimize one number.** When the costs of errors differ by group or context (a false accusation against a person, a safety risk), treat the thresholds as policy decisions with review, not as tuning.

C23.4 Ranking and recommendation: candidate generation, ranking, re-ranking, feedback

A **recommender system** is a system that predicts which items a user will value and ranks them for display. **Ranking** is ordering items by a predicted score so that the most valuable appear first. Real systems have far too many items to score each one for every request, so they work in stages (FW-C23-03):

1. **Candidate generation.** **Candidate generation** is the cheap first stage that narrows a huge catalog to a manageable set of plausible items. Methods include items similar to what the user liked, items from accounts they follow, what is trending, and nearest neighbors in an embedding space. An **embedding** is a list of numbers that represents an item or user so that similar things lie close together; it lets a system find "items like this one" quickly. The aim is recall: do not miss good items. A catalog of 100,000,000 videos might yield about 1,000 candidates.
2. **Ranking.** A heavier model scores each candidate using many features (the user's history, the item's quality, the context) to predict value, and orders them. It might predict several things: the chance of a click, of watching to the end, of a like, of a "not interested" signal, and combine them into one score. The combination weights are product decisions.
3. **Re-ranking.** Rules and adjustments are applied to the ranked list: diversity (not ten videos from one creator), policy (remove harmful items), freshness, business needs, and positions for exploration.
4. **Presentation.** Layout, number of items, explanations ("because you watched..."), and what happens when the user interacts.

Signals. *Collaborative* signals come from the behavior of many users ("people who liked this also liked that"); *content-based* signals come from item attributes (genre, topic, language). Collaborative methods are powerful and need history; content-based methods work for new

items. **Cold start** (C19's cold-start problem, applied to recommendations) is the difficulty for new users and new items, who lack history; defenses are content-based features, popularity within a segment, asking the user a few preferences, and deliberately showing new items to learn (C23.8).

The latency budget (C21.2). All of this runs while the user waits, within a few hundred milliseconds. Scoring cost is the scale limit: scoring 1,000 candidates per request at 20,000 requests a second is 20 million scores a second; at 0.1 millisecond per score that is 2,000 processor-seconds a second, about 2,000 cores; halving candidates halves the cost (Worked example 2). This is why stages exist.

Where a PM intervenes. Better candidates (new sources, languages, fresher items), better labels and objectives (what is the ranking trying to predict?), re-ranking rules (diversity, safety, creator fairness), and presentation (explanations, controls). Each has a different cost, risk and testability.

C23.5 Search and relevance at PM depth: query understanding, ranking signals, quality measures

Search is a recommender where the user states an intent. A search system (FW-C23-04) understands the query (spelling, synonyms, language, what kind of answer is wanted), *retrieves* candidate results from an index, *ranks* them using signals, and presents them.

Kinds of query. *Navigational* ("bank login"): one right answer, and getting it wrong is costly. *Informational* ("why is the sky blue"): many acceptable answers, quality and trust matter. *Transactional* ("buy running shoes"): relevance plus availability, price and trust. *Ambiguous* ("jaguar"): may need diversity or a clarifying prompt. A quality problem often hides in one class, so segment by query type before judging.

Ranking signals. The words and their match to the item's content; the item's quality and authority; freshness; the user's context (location, language, history); and interaction signals (what people click and stay on). Each signal can be gamed or biased, so ask what it really measures.

Measuring quality. Human raters judge result relevance on a scale (for example 0 to 3) using written guidelines; the product then computes measures such as *precision at k* (the share of the top k results judged relevant) and *NDCG* (normalized discounted cumulative gain), which gives more credit for relevant results placed higher: a result's gain is divided by the logarithm of its position, then compared with the best possible ordering. Worked example 4 shows two rankings with equal precision at 5 and different NDCG. Behavior measures complement raters: the share of searches with a click, the share where the user reformulates the query (a sign of failure), the time to a successful result, and abandonment. Raters see quality but not real intent; behavior sees intent but is noisy and biased by position. Use both.

Regional Lens: India. Search and recommendation quality depends on language coverage and on how people actually type. Users often mix languages in one query and write local languages in the Latin alphabet (transliteration), and training data and quality raters are scarcer for some languages. Inferred [unverified: author's synthesis; no source checked]. The consequence for a PM: segment quality measures by language and script, build rater guidelines and test sets for each, and expect cold start to be harsher where data is thin (fall back to content-based signals and popularity within the language). Check current coverage and published language support before assuming parity.

C23.6 Data: quality, bias, label noise, cold start

A model reflects its data. Problems to look for:

- **Selection and exposure bias.** The data contains only what was shown and acted on. If the old system never showed a type of item, the data has no evidence about it, and the model will never learn that people like it (a form of survivorship bias, C12).
- **Position bias.** **Position bias** is the tendency of items shown higher to be clicked more regardless of quality. If the model learns from clicks without correcting for it, it learns that "top positions are good" and reinforces the current order. A common remedy is to randomize positions in a small slice of traffic to measure how much position alone matters.
- **Label noise and proxy labels.** Labels may be wrong (human raters disagree, fraud labels arrive weeks late and only for disputed payments) or may measure something other than what you want (a click is not satisfaction). Ask how the label is produced, how late and how noisy it is, and who is missing.
- **Implicit and explicit feedback.** Implicit feedback (clicks, watch time, skips) is plentiful and ambiguous; explicit feedback (ratings, likes, "not interested") is clearer and sparse and biased toward people who give it. Combine them; treat "not interested" as a strong signal.
- **Drift.** The world changes: tastes, language, tactics of fraudsters. A model decays; monitor it and retrain on fresh data.
- **Privacy and consent.** What may be collected and used (C21.8), and what must be deleted on request.
- **Cold start.** New users and items have no history; handle them explicitly.

Ask: where did this data come from, what does it leave out, and whom does that hurt?

C23.7 Metrics for ranked systems: engagement versus satisfaction, short versus long term, diversity, fairness

A ranked system needs a stack of metrics, not one (FW-C23-05; C05 gave the general method).

Layer	Examples	What it tells you	Danger if alone
Engagement (immediate)	Clicks, plays, time spent	What attracts attention	Rewards clickbait and compulsion (Goodhart, C05)
Satisfaction	Completion, saves, shares, survey ("did you enjoy this?"), low "not interested" rate	Whether people valued it	Surveys are sparse and slow
Long-term	Retention, return frequency, subscription renewal	Whether the product keeps its value	Slow; hard to attribute
Diversity and discovery	Variety of creators and topics, share of new items shown	Health of the catalog and the user's horizon	May cost short-term engagement
Fairness and creator health	Exposure by group, new-creator success	Whether the ecosystem is sustainable and equitable	Needs definitions that fit the context
Guardrails	Reports, complaints, harmful-content rate, latency	Harm and cost	Not a goal in itself; a limit on the others

Short against long term. A change that raises clicks this week may lower retention in a month, because people who feel manipulated leave. Evaluate on long-term measures through a long-running holdout (C14) and tie the primary objective to something people would endorse on reflection. **Choosing the objective.** What the model predicts becomes what the system optimizes: predicting clicks yields clicky results; predicting "watched most of it and did not regret it" yields different ones. Combining predictions with weights is a product decision to be made explicitly and revisited.

C23.8 Feedback loops and unintended effects: popularity bias, filter bubbles, creator incentives

A **feedback loop** is a cycle in which a system's outputs become its future inputs, so that its own choices shape the data it learns from. In a recommender, what is shown gets clicked, clicks train the model, and the model shows more of the same. Effects:

- **Popularity bias.** **Popularity bias** is a tendency for a recommender to favor already-popular items because they have the most data and the most past clicks, which starves new and niche items of exposure and makes the popular more popular (CS-C23-01).
- **Filter bubbles and narrowing.** Personalization can shrink what a user sees to what they already like, reducing discovery and, for some content, exposure to varied views. The research on how strong this effect is, and for whom, is mixed [V: Nguyen, Hui, Harper, Terveen and Konstan, Exploring the Filter Bubble, WWW 2014, pp. 677 to 686: slightly narrowing over time, less narrowing for users who took recommendations; later work disputes the mechanism]; the PM-level point is to measure variety and not assume it.
- **Creator incentives.** Creators adapt to what the system rewards: if clicks win, headlines get more sensational; if watch time wins, videos get longer. The ranking is a policy that shapes supply. Ask what behavior this ranking invites.

- **Metric gaming and abuse.** Bad actors learn the signals and manipulate them (fake engagement, spam).

Exploration. **Exploration** is deliberately showing items whose value the system is unsure of, in order to learn, at some cost to immediate engagement, as opposed to *exploiting* what it already believes is best. A small, controlled exploration budget (for example a few percent of slots) lets new items and new tastes be evaluated. Worked example 5 shows that learning about a new item's click rate takes about a thousand impressions, and that a budget for it is a tiny share of total traffic.

Monitoring the loop (FW-C23-06). Track concentration (share of exposure going to the top items), exposure for new items and creators, variety per user, and long-term retention; and review what the ranking rewards.

C23.9 Experimenting on ranking systems

C14 gave the foundations. Ranking systems add specific problems (FW-C23-07).

Offline against online. **Offline evaluation** is judging a model on historical data without showing it to users. It is fast and cheap and can mislead: it measures how well the model predicts past behavior under the old ranking, which carries the old system's biases, and cannot show how users react to new results, as the Netflix Prize showed (CS-C23-02). Use offline evaluation as a gate (is this worth testing?) and online experiments as the judge.

Pitfalls. *Novelty effects:* a new ranking draws curiosity that fades; read effects by week (C14.4). *Slow effects:* retention and creator behavior change over weeks; use a long-term holdout. *Interference:* users affect each other (in a feed, the content one person posts is seen by others; in a marketplace, a buyer's purchase removes an item), so randomizing users may not isolate the effect; consider cluster randomization or running the test on one side of a market and measuring the other. *Position effects:* when comparing rankings, the difference in positions confounds clicks. *Multiple metrics:* with many metrics some will move by chance (C12.7); fix primary and guardrail metrics in advance (C14.5).

Counterfactual data. To evaluate new rankings from logs, log what was shown and with what probability (including a little randomization), so that you can estimate what would have happened under another policy. Without such logging, you can only guess.

Decision rule. Write it before the test: the primary metric's threshold, guardrail margins, the required duration, and what you will do if results are mixed (C17.4).

C23.10 Answer architecture for "improve the recommendations, search or feed" questions

Section 4 gives the full architecture. In outline: state the goal and who it is for; diagnose the current system by stage (candidates, ranking, re-ranking, presentation) and by segment; propose options at the stage where the problem is; choose metrics with guardrails and a time horizon; plan the experiment; and name the feedback loops and the risks.

C23.11 Worked examples

Section 5 gives five worked examples: choosing a classifier threshold by cost, the stages of a recommendation funnel with a cost estimate, reading a ranking experiment with mixed metrics, comparing search rankings with precision and NDCG, and an exploration budget for new items. Section 11 adds ten questions with model answers and follow-ups.

3. Frameworks

Eight frameworks, set out below. The metric card (FW-C05-01), the experiment decision-rule table (FW-C14-05), the conflicting-metrics readout (FW-C14-07), the launch-or-hold table (FW-C17-04), the latency budget (FW-C21-03), the capacity sheet (FW-C22-03) and the abuse-case pass (FW-C22-08) are reused, not repeated.

FW-C23-01 · The prediction brief · Original

- **Inputs:** a product decision that a prediction could support.
- **Outputs:** the decision and who acts on it; the label and how it is produced; the data and what it leaves out; the cost of each error; the threshold or ranking use; evaluation on unseen, recent and segment data; monitoring and retraining.
- **Assumptions:** a rule would not do the job as well.
- **Tradeoffs:** a model captures subtle patterns and is harder to explain, audit and change.
- **Failure modes:** no decision; a proxy label taken as truth; evaluation only on training data; no monitoring.
- **Worked example:** Worked example 1 and IQ-C23-02.

Item	Answer
Decision and actor	
Label (source, delay, noise)	
Data and gaps	
Cost of a false positive and a false negative	
Evaluation data	
Monitoring and retraining	

FW-C23-02 · Threshold by cost · Original

- **Inputs:** classifier scores, a base rate, costs of each error and a capacity limit for review.
- **Outputs:** a table at several thresholds (flagged, true and false positives, false negatives, precision, recall, total cost), the choice within capacity, and a view of how the base rate changes precision.
- **Assumptions:** costs can be estimated and revisited.
- **Tradeoffs:** a lower threshold catches more and costs more review and more wrong flags.

- **Failure modes:** maximizing accuracy; ignoring capacity; ignoring the base rate; one threshold for all contexts.
- **Worked example:** Worked example 1.

Formulas: **precision** = true positives ÷ flagged; **recall** = true positives ÷ all true; **cost** = false positives × cost₁ + false negatives × cost₂.

FW-C23-03 · The recommendation funnel · Original

- **Inputs:** a catalog, a user context and a latency budget.
- **Outputs:** the stages (candidate generation, ranking, re-ranking, presentation) with, for each, its goal, its signals, its failure modes, its latency share and its cost, and where to intervene.
- **Assumptions:** scoring everything is too expensive.
- **Tradeoffs:** more candidates raise recall and cost; heavier ranking raises quality and latency.
- **Failure modes:** candidates that miss good items; ranking objectives that drift from value; re-ranking rules that no one reviews.
- **Worked example:** Worked example 2.

Stage	Goal	Typical size	Failure	Intervention
Candidate generation	Do not miss good items	100,000,000 to 1,000	Missing new or niche items	New sources, fresher items
Ranking	Order by predicted value	1,000 to 100	Optimizing a proxy	Better labels and objectives
Re-ranking	Diversity, safety, freshness	100 to 20	Rules that no one reviews	Explicit policy and tests
Presentation	Make results usable	20 shown	Confusing layout	Explanations, controls

FW-C23-04 · Search relevance diagnosis · Original

- **Inputs:** a complaint or a metric drop in search quality.
- **Outputs:** the affected query class, the stage at fault (understanding, retrieval, ranking, presentation), measures from raters and behavior, and the fix with its test.
- **Assumptions:** queries can be segmented and rated.
- **Tradeoffs:** raters are accurate and slow; behavior is fast and noisy.
- **Failure modes:** judging overall quality only; ignoring language and query type; trusting clicks with position bias.
- **Worked example:** Worked example 4 and IQ-C23-05.

Steps: **Segment by query type and language** → **Compare rater scores and behavior** → **Locate the stage** → **Fix and test** → **Monitor the class**.

FW-C23-05 · The ranked-system metric stack · Original

- **Inputs:** a ranking system and its goal.
- **Outputs:** metrics by layer (engagement, satisfaction, long-term, diversity and fairness, guardrails), each with its time horizon and its danger when used alone, and the primary metric chosen.
- **Assumptions:** the objective can be tied to what people value on reflection.
- **Tradeoffs:** a long-term metric is trustworthy and slow.
- **Failure modes:** a single engagement metric; no long-term holdout; no guardrails.
- **Worked example:** Worked example 3 and IQ-C23-06.

FW-C23-06 · The feedback-loop check · Original

- **Inputs:** a system whose outputs shape its training data.
- **Outputs:** where outputs become inputs; popularity and concentration measures; exposure for new items and creators; an exploration budget; creator-incentive review; monitoring and a response plan.
- **Assumptions:** exposure and outcomes are logged.
- **Tradeoffs:** exploration costs short-term engagement.
- **Failure modes:** training on clicks from the system's own ranking without correction; no measure of concentration.
- **Worked example:** Worked example 5 and CS-C23-01.

FW-C23-07 · The experiment plan for ranking · Original

- **Inputs:** a ranking change.
- **Outputs:** an offline gate, the online design (unit, duration, interference handling, position handling), primary and guardrail metrics with margins, a novelty read by week, a long-term holdout, logging for counterfactuals, and the decision rule.
- **Assumptions:** the platform can randomize and log.
- **Tradeoffs:** longer tests delay launch and give truer answers.
- **Failure modes:** declaring victory from offline metrics; reading week one; ignoring the creator side.
- **Worked example:** Worked example 3 and IQ-C23-10.

FW-C23-08 · The "improve it" answer spine · Original

- **Inputs:** a prompt to improve recommendations, search or a feed.
- **Outputs:** an answer in seven steps: goal and user; diagnosis by stage and segment; options at the failing stage; metric stack and guardrails; experiment; feedback loops and risks; decision and what would change it.
- **Assumptions:** you can ask clarifying questions.
- **Tradeoffs:** structure shows judgment and can slow the best idea.
- **Failure modes:** "use ML" with no diagnosis; one metric; no loops.

- **Worked example:** the answers in section 11.

4. Answer architecture

The "improve the recommendations, search or feed" answer, for a 30-minute round

Step	Minutes	What to produce
Goal and user	4	What the surface is for, whom it serves, what success looks like for them and the company
Diagnose	6	The current system by stage (candidates, ranking, re-ranking, presentation) and by segment; what data says
Options	6	Two or three changes at the failing stage, with cost and risk
Metrics	5	The metric stack: primary, satisfaction, long term, diversity, guardrails
Experiment	5	Offline gate, online test, novelty and interference, holdout, decision rule
Loops and risks	4	Feedback loops, creator incentives, cold start, fairness; what would change the plan

Pacing rules. If the interviewer names a specific problem (the same ten songs, a drop in search quality), spend the time on diagnosis and options. If you run behind, keep the goal, the stage you would change, the metric stack and the experiment.

How to say it. State the decision a model supports and what it predicts: "The ranking predicts whether someone will watch most of a video; I'd question whether that is what we want." Name the cost of each error before choosing a threshold.

How to close. Two sentences: "I'd change [stage] to [option] because [diagnosis], judge it on [primary metric] with guardrails on [guardrails], and test it for [weeks] with a long-term holdout. The main risk is [feedback loop or incentive], which I'd monitor with [measure]."

Type: improve recommendations

Diagnose by stage; consider exploration and cold start; choose satisfaction and long-term measures beyond clicks.

Type: search quality

Segment by query type and language; use raters and behavior; locate the stage.

Type: classifier or risk score (fraud, spam, harm)

Use the prediction brief and cost-based thresholds; consider a three-zone design with human review.

Type: a ranking change shows mixed metrics

Use the metric stack and the decision table; read effects by week; check guardrails and the long-term holdout.

Type: fairness or bias in a ranked system

Define the exposure or selection rate by group; test for gaps; consider the feedback loop; involve policy and legal.

Common interviewer interventions, and what to do

Interviewer says	What it means	What to do
"What does the model predict?"	Label and objective	Name the label and whether it matches the goal
"The model is 99% accurate."	Base rate	Ask the base rate; give precision and recall
"Clicks went up."	Proxy	Ask about satisfaction and long-term retention
"Why not more personalization?"	Loops	Mention narrowing, concentration and exploration
"How would you test it?"	Experiment plan	Offline gate, online test, novelty, holdout
"New items get no traffic."	Cold start	Content-based signals and an exploration budget
"Is it fair?"	Group effects	Define the measure by group and test it

5. Worked examples

All five examples are **Illustrative**: the products are fictional or use a product type as the setting only, and every figure is an invented assumption (invented). The arithmetic was computed. Each is written as notes you could say aloud.

Worked example 1 — Choosing a classifier threshold by cost (Illustrative)

The situation. A platform's classifier scores 1,000,000 items a day for abuse. One percent are truly abusive: 10,000 bad and 990,000 good (invented). Reviewers can handle 20,000 flagged items a day. The team debates thresholds.

Costs (FW-C23-02). A wrongly flagged good item costs \$2 (a review and some user frustration). A missed abusive item costs \$50 (harm, cleanup, complaints).

Three thresholds.

Threshold	Recall	Precision	Flagged	True positives	False positives	False negatives	Total cost
Strict	60%	80%	7,500	6,000	1,500	4,000	$1,500 \times \$2 + 4,000 \times \$50 = \$203,000$

Threshold	Recall	Precision	Flagged	True positives	False positives	False negatives	Total cost
Medium	80%	50%	16,000	8,000	8,000	2,000	$8,000 \times \$2 + 2,000 \times \$50 = \$116,000$
Loose	95%	20%	47,500	9,500	38,000	500	$38,000 \times \$2 + 500 \times \$50 = \$101,000$

Reading it. On cost alone, the loose threshold is cheapest (\$101,000). But it flags 47,500 items a day and reviewers can handle 20,000; the medium threshold flags 16,000, which fits. Within the capacity limit the medium threshold is the best choice at \$116,000, and it is far better than strict (\$203,000), which misses 4,000 abusive items. The loose threshold would be feasible only with more reviewers: handling the extra 27,500 reviews a day costs staff, and the cost model would have to include that.

A better design than one threshold. Act automatically on the very highest scores (which have high precision), send a middle band to human review, and leave the rest. This raises the value of each reviewer hour.

The base rate trap. The medium threshold wrongly flags 8,000 of 990,000 good items, a false-positive rate of 0.81%. If abuse were rarer, say 0.1% (1,000 bad, 999,000 good), the same classifier would catch 800 and wrongly flag about 8,070: a precision of 9%. Nothing about the classifier changed. The queue would be 90% good items, so the cost of wasted reviews per true catch would rise tenfold, from about \$2 to about \$20. Always ask the base rate before accepting a precision number.

What I'd say. "We'd pick the medium threshold because it minimizes cost within review capacity, and revisit it each quarter as the base rate, the costs and the classifier change. For the top-scoring items we'd act automatically, and I'd measure precision on the automatic band separately."

What this shows. The threshold came from costs and capacity, not from accuracy, and the base rate explained why a precision figure can collapse.

Worked example 2 — The stages of a homepage recommendation (Illustrative)

Setting: a streaming service of the kind Netflix operates. No fact about any real system is claimed.

The situation. A member opens the homepage. The catalog has 15,000 titles (invented). The system must produce rows of titles, personalized, in about 200 milliseconds at the 95th percentile.

The funnel (FW-C23-03). *Eligibility:* remove titles not licensed in the member's country or not suitable for the profile's maturity setting: about 9,000 titles remain. *Candidate generation:* several cheap sources each propose titles: continuing a series the member started, similar to recent watches (embedding neighbors), popular in their country, new

releases in their favorite genres, and titles liked by similar members: together about 400 candidates. *Ranking*: a heavier model scores the 400, predicting the chance the member will start and finish each; it orders them. *Re-ranking and row assembly*: rows are built ("Continue watching," "Because you watched...", "New this week"), with rules: do not repeat a title across rows, mix genres, give new titles a few slots, remove what the member already finished. *Presentation*: artwork, row order, and the number of titles shown.

Latency budget (95th percentile). Fetch features 30 ms; candidate generation 40 ms; ranking 70 ms; re-ranking and assembly 30 ms; rendering and transfer 20 ms; margin 10 ms: 200 ms in all.

Why the stages exist: the cost arithmetic. Suppose 5,000 homepage requests a second at peak and a ranking model that takes 0.2 ms per title. Scoring all 9,000 eligible titles needs $9,000 \times 5,000 = 45$ million scores a second, at 0.2 ms each: 9,000 processor-seconds a second, about 9,000 cores. Scoring 400 candidates needs $400 \times 5,000 = 2$ million scores a second: 400 cores, 22 times less. Candidate generation is what makes the heavy model affordable.

Where a PM could intervene. *Candidates*: a new source for new members. *Ranking*: what the model predicts (starting a title, finishing it, or still being a member in a month). *Re-ranking*: a rule to guarantee variety. *Presentation*: explanations ("Because you watched...").

Cold start. A new member has no history: ask for a few favorite titles at sign-up, use popularity in the country and language, and let the first sessions teach the system; for a new title, use its attributes (genre, cast, language) and a small exposure budget (Worked example 5).

Failure and degradation. If the ranking model is slow or down, fall back to a precomputed popular list per country and show it, with no visible error (graceful degradation, C22.7).

What this shows. Splitting the problem into stages made the heavy model affordable and gave a PM several places to act, each with a different risk.

Worked example 3 — Reading a feed ranking experiment with mixed metrics (Illustrative)

Setting: a social feed of the kind Meta operates. No fact about any real system is claimed.

The situation. A new feed ranking predicts clicks more accurately than the old one. After a four-week test on 5% of users against a control (invented), the readout is:

Metric	Result	95% interval	Guardrail or goal
Clicks per user	+3.1%	+2.6% to +3.6%	Goal
Time spent per user	+1.2%	+0.8% to +1.6%	Goal
28-day retention	−0.15 points	−0.50 to +0.20	Margin −0.30
"I enjoyed my feed" survey score	−1.0 points	−1.8 to −0.2	Margin −0.50
Reports of unwanted content per 1,000 views	+4%	+1% to +7%	Margin +3%

Read it in order (FW-C14-07). *Validity:* the sample ratio and logging checks pass. *Novelty:* clicks by week are +4.5%, +3.1%, +2.2% and +1.8%: the effect is still decaying and has not settled, so the +3.1% overall overstates the long-run effect, perhaps by half or more (C14.4). *Guardrails:* retention's lower bound (−0.50) is outside its margin (−0.30) though the interval includes zero (uncertain, not proven harm); the survey score is clearly lower, with the whole interval below zero and its lower end beyond the margin; reports of unwanted content are up, with the interval partly beyond the margin. *Goals:* clicks and time are up.

The diagnosis. The model improved at the thing it was trained on (clicks) and the system moved toward content that draws clicks and feels worse. Higher engagement together with a lower self-reported enjoyment is the classic pattern of optimizing a proxy (Goodhart, C05).

Decision (FW-C17-04, FW-C23-07). Do not launch as is. Investigate which content types and which segments drove the click gain and the survey drop; change the objective, for example by combining the click prediction with a prediction of "valued" (a survey-based or "not interested"-based signal) and a penalty for content that draws reports; and retest for six weeks with a long-term holdout of 1% to check retention. Set the decision rule in advance: launch if the survey score is no worse than −0.2 and retention's lower bound is above −0.30.

What I'd tell the team. "The model predicts clicks better. It does not follow that people are better served. We'll keep the gain where people also say they value it."

What this shows. A mixed readout is resolved by reading novelty, guardrails and the objective, not by adding up the wins.

Worked example 4 — Comparing search rankings: precision and NDCG (Illustrative)

Setting: a web search product of the kind Google offers. No fact about any real system is claimed.

The situation. Two rankings are compared for one query. Raters judge each of the top five results on a 0-to-3 scale (3 is perfect) (invented). Ranking A returns grades [3, 2, 3, 0, 1] in order; ranking B returns [3, 3, 2, 1, 0].

Precision at 5. If a grade of 2 or 3 counts as relevant, A has three relevant results in five (0.6) and B has three (0.6): equal.

NDCG. Discounted cumulative gain adds each result's grade divided by $\log_2(\text{position} + 1)$, so a good result at position 1 counts fully and one at position 5 counts less. A: $3 \div \log_2 2 + 2 \div \log_2 3 + 3 \div \log_2 4 + 0 + 1 \div \log_2 6 = 3 + 1.26 + 1.5 + 0 + 0.39 = 6.15$. B: $3 + 3 \div 1.585 + 2 \div 2 + 1 \div 2.32 + 0 = 3 + 1.89 + 1 + 0.43 + 0 = 6.32$. The best possible ordering of these grades is B's, so B's normalized score is 1.00 and A's is $6.15 \div 6.32 = 0.97$. B is better because it puts its second-best result above its weaker ones.

Why a PM cares. Two rankings with the same share of relevant results can differ in order, and users look at the top. A measure that ignores position would miss it.

Is a difference real? Over a sample of 300 queries, suppose B beats A by 0.03 in average NDCG, with the standard deviation of per-query differences at about 0.15: the standard error is $0.15 \div \sqrt{300} = 0.0087$, so the 0.03 difference is about 3.5 standard errors: unlikely to be chance. Also check that raters agree (if two raters disagree on 20% of results, average their grades) and that the gain holds in each language and query type (a gain in English queries may hide a loss in others).

Add behavior. Compare click-through, the share of searches with no click, and the share where users reformulate. If NDCG rises and reformulations also rise, investigate before launching.

What this shows. Position-aware measures and a significance check turned "looks better" into a defensible comparison.

Worked example 5 — An exploration budget for new items (Illustrative)

Setting: an online store of the kind Amazon operates. No fact about any real system is claimed.

The situation. A recommender trained on past clicks rarely shows new products, because they have no clicks. Sellers add 5,000 new products a day (invented). The store shows about 2,000,000,000 recommendation impressions a day, with an average click rate of 6%.

How much exposure does a new item need? To estimate a new item's click rate of about 5% to within a useful margin, 1,000 impressions give a standard error of $\sqrt{(0.05 \times 0.95 \div 1,000)} = 0.0069$, so a 95% interval of about ± 1.35 points: enough to tell a 5% item from a 2% one, not from a 4% one.

The budget (FW-C23-06). 5,000 new items $\times$ 1,000 impressions = 5,000,000 impressions a day, which is $5,000,000 \div 2,000,000,000 = 0.25\%$ of all impressions.

The cost. If new items average a 3% click rate against 6% for established ones, the lost clicks are $5,000,000 \times (0.06 - 0.03) = 150,000$ a day, which is $150,000 \div (2,000,000,000 \times 0.06) = 0.125\%$ of all clicks. The price of learning about every new product is about an eighth of one percent of clicks.

The benefit. Items that turn out good rise in the ranking; the catalog stays fresh; new sellers have a chance, which keeps supply healthy (a marketplace concern, C19). Without exploration the system would show only items it already knows, and new products would never earn the data to be shown.

Design details. Show the exploratory impressions in lower-risk slots and spread them over users; stop showing an item once the estimate is clear (it is good or bad); weight toward items with strong content signals (category, price, images); and watch guardrails (complaints, returns) because new items are riskier. Count exploration impressions separately from normal ones so that evaluation does not confuse them.

What this shows. A small, computed budget made exploration affordable and measurable, and showed that the cost of learning is tiny against the cost of a catalog that never changes.

6. Baseline, Strong, Standout

Prompt: "Your marketplace's 'recommended for you' module gets many clicks but few purchases. What do you do?"

Baseline answer

"I'd train a better model, maybe a deep learning model, on more data so that recommendations are more relevant, and add more features like browsing history."

What is wrong: a model change with no diagnosis; clicks and purchases are not compared; no mention of what the model predicts or how a fix would be measured.

Strong answer

"First I'd find out why clicks do not turn into purchases. A likely cause is that the model is trained to predict clicks, so it learns to show items that attract clicks (attractive images, low prices) and not items people want to buy. I'd check what the label is, and break the funnel: click, view, add to cart, purchase, by segment and by position."

"Options: change the objective to predict purchase or add-to-cart (with fewer labels, so I'd combine signals), filter out items that are out of stock or poor value, and rerank for variety. I'd measure purchase conversion and revenue per session as the primary measures with return rate and satisfaction as guardrails, and test the change against the current module."

Standout answer

Everything in the Strong answer, plus:

"I'd look at where the funnel breaks: if people click and then leave within seconds, the items look good and are not what they want, a mismatch between the label and the goal. I'd check position bias, since top slots draw clicks regardless of fit, by randomizing positions in a small slice to measure how much position alone explains."

"I'd think about the data: purchases are rarer than clicks, so a purchase-trained model has less data and delayed labels; a combined objective, weighting purchase heavily and click lightly, is a product decision I'd make explicit and revisit. I'd check whether sponsored or high-margin items are being promoted by the ranking, since that would explain clicks without purchases and raise a trust question."

"For evaluation, an offline gain in predicting purchases is a gate, not a verdict. I'd run an online test for at least two to four weeks to see novelty fade, with a holdout, and guardrails on returns and complaints. I'd check the effect on new items and sellers, because a purchase-only objective can starve new products, so I'd keep a small exploration budget."

"What would change my plan: if the funnel shows the drop is after the click (price, delivery, stock), the ranking is not the problem and the fix belongs elsewhere."

What moves an answer up a band

From Baseline to Strong	From Strong to Standout
Diagnoses with the funnel and questions the label	Checks position bias, label delay and incentives such as sponsored items
Proposes an objective change with guardrails	Uses an offline gate, a multi-week online test and a holdout
Names primary and guardrail metrics	Considers new items, sellers and when the ranking is not the cause

7. Company lens

All claims describe what companies say publicly or what candidates commonly report, not insider knowledge. Interview formats change [as of 2026-10].

Company lens: Google [as of 2026-10]. Google publishes guidelines used by human raters to judge search quality, and describes search as understanding queries, retrieving and ranking Public [V: Google Search Quality Evaluator Guidelines]. Candidates commonly report prompts on search relevance, ranking changes and evaluation Inferred. Practical consequence: separate query types and languages, use raters and behavior together, and talk about position-aware measures and trust.

Company lens: Meta [as of 2026-10]. Meta has publicly described changes to feed ranking, including a 2018 shift toward interactions between people, and has published information about how ranking works Public [V: Facebook newsroom, 11 January 2018]. Candidates commonly report prompts on feed ranking, engagement versus wellbeing and ranking experiments Inferred. Practical consequence: separate engagement from satisfaction and long-term effects, read novelty by week, and treat guardrails and creator effects as part of the answer.

Company lens: Microsoft [as of 2026-10]. Microsoft owns a professional network with job and people recommendations and has published responsible-AI principles and standards Public [V: Microsoft's June 2016 announcement of the LinkedIn acquisition; Responsible AI Standard shared publicly in 2022]. Candidates commonly report prompts on recommendations with fairness and enterprise considerations Inferred. Practical consequence: define fairness measures by group, check feedback loops, and involve policy and legal early.

Company lens: Amazon [as of 2026-10]. Amazon's researchers published an influential paper on item-to-item collaborative filtering for recommendations in 2003 Public [V: Linden, Smith and York, IEEE Internet Computing 7(1), 2003], and the store mixes recommendations with sponsored placements. Candidates commonly report prompts on product recommendations, new-item cold start and seller effects Inferred. Practical consequence: reason about labels (clicks versus purchases), exploration for new products, and trust when paid placements share the page.

Company lens: Apple [as of 2026-10]. Apple describes doing machine learning on the device and using privacy-preserving techniques Public [V: Apple privacy and support pages]. Candidates commonly report prompts on personalization under privacy constraints and on music or media recommendations Inferred. Practical consequence: state what data the personalization needs and where it is processed, and offer controls and explanations.

Company lens: Netflix [as of 2026-10]. Netflix has published papers and blog posts on its recommendation approach, including the point that recommendations drive a large share of viewing and its 2006 prize contest Public [V: Netflix Tech Blog, Beyond the 5 Stars; Netflix Prize 2006]. Candidates commonly report prompts on homepage personalization, evaluation and experimentation Inferred. Practical consequence: judge recommendations on whether people start, finish and keep valuing the service, run long tests, and treat offline accuracy as a gate.

Company lens: Stripe [as of 2026-10]. Stripe offers machine-learning-based fraud screening as a product, and payments companies must balance blocked fraud against blocked good customers Public [V: Stripe pricing page and product descriptions (Radar machine learning)]. Candidates report prompts on risk scoring and thresholds Inferred. Practical consequence: set thresholds by the cost of each error, design review zones, and measure the cost of false declines.

8. Common mistakes

#	Mistake	How to detect it	Fix
1	Model with no decision	"Use ML to improve it"	State the decision, label and cost of errors
2	Accuracy for a rare event	"99% accurate"	Give base rate, precision and recall
3	One threshold by habit	A default of 0.5	Choose by cost and capacity; consider zones

#	Mistake	How to detect it	Fix
4	A single engagement metric	Clicks only	Use the metric stack with long-term and guardrails
5	Offline win as launch case	"Better AUC"	Treat as a gate; run an online test
6	Ignoring position bias	Learning from clicks on the current order	Randomize a slice; correct for position
7	Ignoring cold start	New items and users get nothing	Content signals and an exploration budget
8	Ignoring feedback loops	The system trains on its own output	Measure concentration; explore; monitor
9	Reading week one	A novelty bump taken as the effect	Read by week; long-term holdout
10	Ignoring creators and groups	Only average user effects	Check creator exposure and group-level measures

9. Mental models applied

Four models, each tied to a decision in this chapter. [\[\[goodharts-law\]\]](#) is from C02 and [\[\[base-rate\]\]](#) from C12.

Goodhart's law (MM-goodhart, see [\[\[goodharts-law\]\]](#), C02). When a measure becomes a target it stops being a good measure. A ranking trained to maximize clicks drifts toward click-attracting content. The defense is a stack of metrics, an objective tied to value on reflection, and guardrails (Worked example 3).

Explore and exploit (MM-explore-exploit). A decision-maker must choose between using what it believes is best and trying options to learn; too much exploitation freezes learning and too much exploration wastes value [V: Sutton and Barto, Reinforcement Learning: An Introduction, 2nd edition, MIT Press, 2018, chapter 2 on multi-armed bandits]. In a recommender it is the exploration budget (Worked example 5).

The Matthew effect (MM-matthew-effect). Those who have advantage gain more of it ("the rich get richer") [V: Merton, The Matthew Effect in Science, Science 159(3810), 5 January 1968, pp. 56 to 63]. In ranking it is popularity bias: popular items get exposure, exposure creates popularity. Name it when you see concentration rising.

Base rates (MM-base-rate, see [\[\[base-rate\]\]](#), C12). The rarity of what you are looking for shapes every precision figure. Always ask how common the thing is before judging a classifier (Worked example 1).

Engineering, design and data-science perspectives

- **Engineering and ML engineering.** They build and monitor the model and the serving system. Ask what the model predicts, what data it uses and how fresh, how it is evaluated

offline, what the latency and cost are, and what the fallback is when it fails. Agree the logging you need for counterfactual evaluation before launch.

- **Design and research.** Designers decide how recommendations are explained and how users give feedback ("not interested," controls to reset). Research shows what people actually value, which is the source of better labels and survey measures.
- **Data science.** They own metric definitions, experiments and the diagnosis of changes. Ask for effects by week, by segment and by creator side, for the long-term holdout, and for how interference is handled.
- **Policy, legal, trust and safety.** They define harms, fairness requirements and what content may be promoted. Bring them in when a ranking change touches sensitive content, groups of people or creators' livelihoods.

Checklist: before you propose a change to a ranked system

1. Have I said what the model predicts, from what label, and whether that matches the goal?
2. Do I know the base rate, the cost of each error and the review capacity before choosing a threshold?
3. Can I describe the system by stage and say where the problem is?
4. Do I have a metric stack with satisfaction, long-term and guardrail measures, not just clicks?
5. Have I named data problems (position bias, label noise, selection bias, cold start)?
6. Have I considered feedback loops, popularity bias and creator incentives, and an exploration budget?
7. Is the experiment plan written (offline gate, novelty by week, interference, long-term holdout, decision rule)?
8. Have I said what would change my recommendation?

10. Case studies

CS-C23-01 · The music app that learned from its own top 50 · Illustrative · Beginner

Fictional composite. Every number is invented (invented).

Situation. A music app with 8,000,000 tracks recommends songs on its home screen. The recommender is trained on listening data: songs that were played are positive examples. Six months after a retrain, product managers notice that the top 50 songs account for 62% of all plays, up from 40% when the system launched, and that new releases get only 1.5% of home-screen impressions. Independent artists complain that no one hears their songs.

Constraints. A model trained on the app's own plays, a home screen with limited slots, and a metric (plays per user) that is rising.

Data. Plays per user are up 4% over six months. The share of listeners who play at least one song they had never heard before in a week has fallen from 31% to 22%. Thirty-day retention is flat.

Problem definition. As an observable problem: "The system recommends what is already popular, so popular songs gain plays and new songs get none, and plays per user rise while discovery falls."

Analysis. A feedback loop (FW-C23-06): what is shown gets played, plays train the model, and the model shows more of the same. Popularity bias concentrates exposure; new songs have no plays, so they are never shown (cold start). The plays metric hides this because familiar songs are easy to play. Exploring has a cost: songs the system is unsure of are skipped more often. Suppose an exploratory slot is skipped 55% of the time and a normal slot 25% of the time. If 8% of slots are exploratory, the overall skip rate rises by $0.08 \times (0.55 - 0.25) = 2.4$ points.

Options.

Option	Optimizes	Costs
A. Leave it; plays are rising	Short-term plays	Discovery keeps falling; artists leave; long-term risk
B. Reserve 8% of home-screen slots for exploration, with new releases and unfamiliar songs matched to taste	Discovery and fresh data	A higher skip rate on those slots (about 2.4 points overall)
C. Remove popularity from the model	A flatter distribution	Worse recommendations for most listeners

Tradeoffs. B costs a small, measurable drop in immediate smoothness and buys learning and discovery. A is easy and reinforces the loop. C swings too far.

Decision. Choose B, with exploration slots chosen by content similarity to taste, and a guardrail on skip rate and session length. Measure the top-50 share, the share of listeners who discover a new song in a week, new-release impressions, retention at 90 days and the skip rate. Run it as an experiment with a holdout for eight weeks.

Outcome. In this fictional scenario, after eight weeks the top-50 share fell from 62% to 55%, new-release impressions rose from 1.5% to 6%, the share of listeners discovering a new song in a week rose from 22% to 29%, the overall skip rate rose 2.3 points, and 90-day retention was 0.8 points higher than the holdout (an interval of +0.2 to +1.4). The team kept the budget and added it to its dashboards.

Alternative outcomes. If retention had not moved, discovery would still be valuable for the artists' side of the platform, and the team would weigh the skip cost against it. If the skip rate had risen by 6 points, the exploratory slots would have been too poorly matched and the team would have tuned the candidate source. The lesson: a metric that rewards the familiar will build a system that only shows the familiar; measure discovery too.

CS-C23-02 · The Netflix Prize, 2006 to 2009: an accuracy contest and what shipped · Verified · Advanced

Sources (types): Netflix's own engineering account, "Netflix Recommendations: Beyond the 5 stars (Part 1)" by Xavier Amatriain and Justin Basilico (Netflix Tech Blog, 6 April 2012), read in full, a primary source for what Netflix did with the contest's results; the Associated Press report of the award (21 September 2009), for the winners and the number of contestants; the contest's results as summarized in reference coverage (Wikipedia) and a statistics paper on the contest (Feuerverger, He and Khatri, Statistical Science, 2012), for the data set and the number of contestants. The analysis is the author's. The sources do not reveal Netflix's internal evaluation beyond what its post says. Re-read in October 2026.

Situation. In October 2006 Netflix launched a public contest, the Netflix Prize, offering \$1 million to the first team to improve the accuracy of its movie recommendation system, Cinematch, by 10%. Netflix said it needed a proxy question that was easier to evaluate: the root mean squared error (RMSE) of predicted star ratings, where Cinematch scored 0.9525 on the test set and the winning score had to be 0.8572 or lower. By the end the Associated Press reported more than 51,000 contestants from 186 countries

[V: AP, 21 September 2009]. A year into the contest the first Progress Prize went to the team Korbell (BellKor), whose solution combined 107 algorithms, after more than 2,000 hours of work by the team's own report, for an 8.43% improvement, and the team gave Netflix its source code [V: Netflix Tech Blog, 2012]. Netflix said that the two best-performing algorithms in that ensemble, a matrix-factorization method (0.8914 RMSE on its own) and a restricted Boltzmann machine (0.8990 on its own; a linear blend of the two gave 0.88), were put into production and were still in use in April 2012, after work to overcome limitations: they were built for the 100 million ratings of the contest data, not the more than 5 billion Netflix had, and were not built to adapt as members added ratings. On 21 September 2009 the grand prize was awarded in New York to BellKor's Pragmatic Chaos, a seven-member team formed by merging earlier teams, for an improvement reported as 10.06% [V: AP; reference summary]; on the public leaderboard it and a rival team, The Ensemble, stood at about 10.09% and 10.10%, and BellKor's Pragmatic Chaos won by submitting its final entry minutes earlier [V: AP]. Netflix described the grand-prize ensemble as blending hundreds of predictive models, said that it evaluated some of the new methods offline but that the additional accuracy gains it measured did not seem to justify the engineering effort needed to bring them into production, and noted that its focus had shifted and that its business had moved toward streaming (launched in 2007), which changed how data was collected and recommendations were delivered [V: Netflix Tech Blog, 2012].

Constraints. A contest dataset that was a small, frozen sample of ratings; a production system with orders of magnitude more data that changed continuously; a proxy metric (error in predicted ratings) standing in for the product's actual goal; and a business changing from mailing discs to streaming.

Data. As reported: the dates; the prize and target; the proxy metric; the baseline and target RMSE; the number of participants; the 2007 and 2009 results; Netflix's statement about what was and was not deployed and why. Contest scores were computed on a public "quiz" subset, where Cinematch scored 0.9514, and on a hidden test set, where it scored 0.9525; Netflix's post quotes the test-set figures [V: Netflix post; reference summary].

Problem definition. *The author's reading.* The company wanted better recommendations, and it measured an easier thing: accuracy at predicting ratings. As an observable problem: "An improvement on the proxy metric was large and did not translate in full to the product."

Analysis. *The author's analysis, through this chapter's tools.*

The proxy. Predicting the star rating a member would give a movie is not the same as choosing which titles to put in front of them. Ratings were explicit and sparse; the product's key decisions concern ranking a few titles out of thousands, where the top of the list matters most (C23.5's lesson on position-aware measures). A small gain in rating error need not change what appears on the screen. This is the proxy-worship risk of C23.1, and Netflix itself, by its account, used the contest to find methods and then evaluated them against the production problem.

Offline against online (C23.9). The contest was an offline evaluation on a fixed dataset. Netflix's reported decision to adopt a blend of two methods and not the winning ensemble shows offline results used as a gate: the cost of building and maintaining a 100-algorithm ensemble was weighed against measured gains.

Scale and adaptivity. Methods developed for 100 million ratings had to be reworked for 5 billion and for members who add ratings continuously: engineering constraints that a contest does not test (C21, C22).

The data changed. A move to streaming replaced explicit ratings with implicit signals (what members play, how long, when they stop), which are more plentiful and noisier, and shifted the problem from predicting ratings to ranking what to play next. The question the contest answered was becoming less central.

What the evidence does not show. Netflix's internal measurements of the gain from each method, the exact production blend's effect on retention, and how its metrics evolved. The case supports an analysis of proxies and offline evaluation, not a judgment about the contest's cost or value.

Options. *The author's reconstruction of choices a team could face after the contest; not a record of Netflix's deliberations.*

Option	Optimizes	Costs
A. Deploy the full grand-prize ensemble	The contest's measured accuracy	Large engineering and maintenance cost; a gain on a proxy
B. Deploy the simpler components that gave most of the gain, adapted to production data	Value per engineering effort	Gives up the last increments of accuracy
C. Redefine the objective around streaming behavior and ranking, and invest there	Fit with the product	Needs new data, metrics and experiments

Tradeoffs. A buys accuracy at a high cost and on a proxy. B captures most of the gain more cheaply. C is the most relevant and the most work, and is where a business that has changed its product must go.

Decision. As reported, Netflix deployed a blend of two methods from the 2007 solution, did not deploy the grand-prize ensemble, and shifted its recommendation work toward the streaming product.

Outcome. Reported: the winning score, the partial deployment, the stated reason, and, in Netflix's later publications, an emphasis on ranking and on testing with members. Financial results of the contest are outside the sources used here.

Alternative outcomes. *Inference.* Had the ensemble been deployed, the maintenance burden might have slowed later work for little product gain; had no contest been held, the company would not have had the matrix-factorization methods that became standard. A reader should avoid hindsight: a public contest was a cheap way to find methods. The method to carry away: know what your metric stands in for, treat offline gains as a gate with a price on complexity, test on the production problem with real members, and expect the product's data and objective to change.

11. Interview questions and model answers

Ten questions. Model answers are **Illustrative** (composite candidates), every figure is an assumption, and the arithmetic was computed. Each ends with follow-ups and answers. Where a real company's product is the setting, the answer reasons from public information and states no internal fact. The answers are fuller than a first pass in a real round.

IQ-C23-01 · Beginner · ai · AI

Prompt. Explain in plain terms how a recommendation system decides what to show you.

Rubric.

- **Baseline:** "It uses an algorithm that learns what you like."
- **Strong:** Describes candidate generation, ranking and re-ranking, and what data it learns from.
- **Standout:** Explains why stages exist, what the model predicts and how that choice shapes results, mentions cold start and feedback loops, and keeps it plain.

Model answer. Think of a very large store and a very small shop window. The system has millions of items and room to show twenty, so it works in steps.

First it narrows the store to a few hundred plausible items, cheaply: things similar to what you liked, things from people you follow, what is popular where you are, what is new in your favorite topics. This step is about not missing good things.

Then a more careful model scores each of those few hundred and puts them in order. The model has learned from past behavior: what people like you watched, finished, liked or skipped. What it predicts matters: if it predicts clicks, the order favors things that attract clicks; if it predicts "watched most of it and did not regret it," the order favors something else.

Then rules adjust the list: do not show ten items from one creator, remove anything unsafe or already seen, give new items a small chance. Finally the screen decides how to present them.

Two things make it hard. A new user or new item has no history (the cold start), so the system leans on item attributes and popularity until it learns. And the system learns from what it showed, so it can keep showing more of the same: a feedback loop that product teams must watch.

Follow-ups.

- **Q.** Why not score every item for every user? **A.** "The cost. Scoring millions of items per request with a heavy model would need far more computers and time than the user will wait, a few hundred milliseconds. The staged design keeps the heavy model for a few hundred candidates and spends far less."
- **Q.** What does "the model predicts clicks" imply for the product? **A.** "It will tend to promote whatever draws clicks, including sensational or misleading items, because that is what it is rewarded for. A product team can change the target (for example, include completion or satisfaction signals) and should check that what the model optimizes is what people value."

IQ-C23-02 · Beginner · ai · AI

Prompt. A fraud model is 99% accurate. Is that good?

Rubric.

- **Baseline:** "Yes, 99% is excellent."
- **Strong:** Asks the base rate and uses precision and recall.
- **Standout:** Shows with numbers that a useless model can match the accuracy, relates it to cost and thresholds, and says what to ask for instead.

Model answer. Not necessarily; accuracy alone says little when the thing you are looking for is rare. Suppose 1% of 1,000,000 transactions are fraud: 10,000 fraudulent and 990,000 legitimate. A model that labels everything legitimate is right 990,000 times: 99% accurate, and catches no fraud at all (recall 0%).

Now take a useful model that catches 8,000 of the 10,000 frauds (recall 80%) and wrongly flags 8,000 legitimate transactions. Its correct decisions are the 8,000 frauds caught plus the 982,000 legitimate transactions it left alone: 990,000, so it is also 99.0% accurate. The same accuracy describes a model that does nothing and a model that catches 80% of fraud. That is why accuracy is the wrong measure here.

What I would ask for instead: the base rate; precision (of what we flag, how much is fraud: here 50%) and recall (of all fraud, how much we catch: 80%); the cost of each error (a blocked good customer against a missed fraud); and performance at the threshold we plan to use, on recent data and by segment. Then I would decide whether the model is good by whether its total cost is lower than the alternatives within our review capacity.

Follow-ups.

- **Q.** What if the base rate drops to 0.1%? **A.** "Precision falls sharply even if the model is unchanged: with the same 80% recall and about a 0.8% false-positive rate, it catches 800 frauds and flags about 8,000 good transactions, a precision of about 9%. The queue becomes mostly good transactions, so cost per fraud caught rises about tenfold, and the threshold should be revisited."
- **Q.** Why not just report recall? **A.** "Recall alone rewards flagging everything, which has perfect recall and useless precision. The pair, with the cost of each error, shows the tradeoff."

IQ-C23-03 · Intermediate · ai · AI · Apple

Prompt. Improve the recommendations in a music app for a user who plays the same ten songs.

Rubric.

- **Baseline:** "Recommend more songs that are similar to those ten."
- **Strong:** Recognizes that familiarity is rewarded and proposes some exploration and a way to measure it.
- **Standout:** Diagnoses whether the user wants novelty, designs a controlled exploration slot, defines satisfaction and long-term measures, considers privacy and controls, and plans a test.

Model answer. The setting is general; I know such products only from public information, and nothing here describes any real design. The first question is whether this is a problem. A person who plays the same ten songs may be perfectly satisfied (a workout list, a comfort album). The aim is to help the people who would like more, without annoying those who would not.

Diagnose. Which of these users are stuck (they have skipped recent suggestions, searched for new music elsewhere, or their listening has narrowed over months) and which are content? A recommender trained on plays will favor the familiar songs because plays are what it was rewarded for; that is a feedback loop.

Options. (1) *Blend:* keep the familiar songs and add adjacent ones (same mood, tempo or artist's other work), a small step from taste, in a clearly labeled "something new" slot. (2) *An exploration budget:* a small share of slots, say 8%, chosen to learn. (3) *User control:* a "more like this / less like this" signal and a reset, which also respects preference. (4) *A separate discovery surface* so that the main screen can stay familiar for those who want that.

Cost. Exploratory slots are accepted less often than familiar ones. Suppose (invented) a new song is played through 30% of the time against 85% for a familiar one: 8 exploratory slots per 100 lose about $(0.85 - 0.30) \times 8 = 4.4$ completed plays per 100 slots in the short term, and yield about 2.4 accepted new songs. The product must decide that the discovery is worth the short-term cost for users who show a desire for it.

Metrics. Primary: new songs added to a library or played to completion at least twice (a satisfaction signal). Guardrails: overall skip rate and session length, "not interested" feedback. Long term: 90-day retention against a holdout. Segment by user type, since the effect differs.

Privacy. Personalization uses listening history; state what is used and where it is processed, and let users clear it.

Test. A multi-week test with a holdout, reading effects by week for novelty, and by user segment.

Company note. Apple is the setting only: Inferred from candidate reports of media-recommendation prompts [unverified: candidate-reported]. No claim is made about any real product.

Follow-ups.

- **Q.** How do you know whether a user wants novelty? **A.** "Behavior: skipping the same songs, searching for songs outside the ten, narrowing listening over time compared with their own history, and explicit signals (a 'show me something new' tap). I'd start with those users, and offer others a control and not a change."
- **Q.** What if the exploration slots lower the skip-rate guardrail? **A.** "Make exploration better matched: choose adjacent songs, place them after a familiar one, and reduce the share until the guardrail holds. If no setting gives a net benefit for a segment, leave that segment on the familiar experience."

IQ-C23-04 · Intermediate · ai · AI · Netflix

Prompt. How would you measure whether homepage personalization is working, beyond clicks?

Rubric.

- **Baseline:** "Click-through rate on recommended titles."
- **Strong:** Uses plays and completion and retention.
- **Standout:** Explains why clicks mislead, defines qualified plays, a layered stack with short and long term and diversity, plans holdouts, and notes novelty and offline gates.

Model answer. The setting is general; nothing here describes any real measurement. Clicks measure curiosity about a thumbnail, not whether the title was worth watching. A better artwork or a catchy title can raise clicks and lower satisfaction.

Suppose (invented) a change raises clicks on recommended titles by 5%, and 40% of the plays it adds are abandoned within two minutes. The extra plays are worth little. So I'd define a *qualified play*: a play that reaches a set point, for example 70% of an episode or a second episode. If qualified plays rose only 0.5%, the real gain is a tenth of the click gain.

The stack (FW-C23-05). Immediate: play starts and the time to first play (how quickly a member finds something). Satisfaction: qualified plays, completions, ratings or thumbs where given, and the share of members who say they found something to watch. Long term:

retention and return frequency at 28 and 90 days, which is where the value of recommendations shows. Diversity and discovery: the variety of titles and genres watched, the share of plays of titles the member has not seen the like of before, and exposure for newer titles. Guardrails: search use (a sign that recommendations failed), complaints, cost.

Experiments. Offline accuracy is a gate; the judge is an online test with a holdout, run for weeks to let novelty fade, with a long-term holdout for retention. Analyze by member segment (new members, heavy viewers, kids' profiles).

What I'd watch for. A rise in clicks with a rise in search (members are not finding what they want on the homepage), and concentration (most plays from a few titles).

Company note. Netflix is the setting only: Inferred from candidate reports of personalization and measurement prompts [unverified: candidate-reported]. No Netflix data or practice is claimed.

Follow-ups.

- **Q.** Why is retention the right long-term measure? **A.** "A subscription's value depends on members continuing to find enough they enjoy; recommendations are meant to make that easier. If a change raised plays and members still left at the same rate, the system was not helping them. Retention is slow and noisy, so use a long holdout and combine it with earlier satisfaction signals."
- **Q.** How do you avoid optimizing for easy, short content? **A.** "Measure satisfaction per qualified play and long-term retention, not total plays, and check the mix of what is watched. If the model's predicted 'finish' favors short items, add a measure of how much members valued what they finished."

IQ-C23-05 · Intermediate · ai · AI · Google

Prompt. Search results got worse for a class of queries. How do you find out why and what to do?

Rubric.

- **Baseline:** "Retrain the ranking model."
- **Strong:** Segments by query type and checks stages.
- **Standout:** Uses raters and behavior together, shows how an overall average can hide a class problem, locates the stage, handles language and freshness, and plans a fix with a test and monitoring.

Model answer. The setting is general; nothing here describes any real system. First, confirm and size the problem: which queries, how much worse, since when.

Why the average hides it. Suppose (invented) a class, say recipe queries, is 6% of traffic, and precision at 5 for the class fell from 0.72 to 0.64, a drop of 0.08. The overall figure moves by $0.06 \times 0.08 = 0.0048$, from 0.800 to 0.795: invisible in the average. So I segment by query type, topic, language and device before I trust any overall number.

Diagnose by stage (FW-C23-04). *Understanding:* is the system misreading these queries (spelling, synonyms, a language or script issue)? *Retrieval:* are good results absent from the candidates (an index or freshness problem)? *Ranking:* are good results retrieved but ranked low (a new signal or a model change)? *Presentation:* did a layout change bury results? Compare before and after on a rated sample, and look at what changed on the date the quality fell (a ranking update, an index change, a new data source).

Use two kinds of evidence. Human raters judging results on a 0-to-3 scale with NDCG (position-aware) show quality; behavior (clicks, reformulations, abandonment) shows intent. If raters say results are worse and reformulations rose, the problem is real. If only clicks fell, a layout change may explain it.

Fix. Depending on the stage: repair the index or freshness; restore a signal that the update dropped, or add a feature for the class; adjust the ranking for that class; then test. Roll back the change if it clearly caused the drop (C17).

Monitor. Add the class to the standing quality dashboard so that the next regression is seen early, and add raters' test queries for it.

Company note. Google is the setting only: `Inferred` from candidate reports of search quality prompts `[unverified: candidate-reported]`. No Google data or practice is claimed.

Follow-ups.

- **Q.** How do you tell a ranking problem from a retrieval problem? **A.** "Look for a known good result for a sample of the failing queries: if it is not in the candidate set, retrieval or the index is at fault; if it is retrieved but ranked low, ranking is at fault. Raters or a hand-picked list of good results make the test concrete."
- **Q.** What if raters and clicks disagree? **A.** "Investigate before acting: raters may misjudge intent, and clicks are biased by position and layout. Look at a few disagreements in detail, check whether the layout changed, and prefer a controlled experiment to settle it."

IQ-C23-06 · Hard · ai · AI · Meta

Prompt. A new feed ranking raises time spent by 5%, but users say they feel worse. What do you do?

Rubric.

- **Baseline:** "Ship it; time spent is up," or "kill it."
- **Strong:** Treats the survey as a guardrail and investigates the cause.
- **Standout:** Distinguishes valued from unvalued time, finds which content and segments drive it, changes the objective, reads novelty and retention, plans a longer test with a decision rule, and weighs creators and harm.

Model answer. The setting is general; nothing here describes any real system. Time spent is an engagement measure; how people feel is a satisfaction and wellbeing measure. When they diverge, I do not average them; I find out why, because the system may be succeeding at the wrong thing (Goodhart).

Investigate. (1) *Composition*: which content types, creators and formats account for the added time, and what do people say about those types? (2) *Segments*: who feels worse (teens, heavy users, new users)? Averages can hide a group that is harmed. (3) *Time shape*: is the +5% a novelty bump? Read effects by week; suppose weekly gains are +7%, +5%, +4% and +3.5%: still falling, so the settled effect is smaller. (4) *Long term*: retention and return frequency at 28 days against a holdout. (5) *Other guardrails*: reports of unwanted content, "not interested" taps, complaints.

Valued time. Suppose (invented) the survey shows a 1.5-point drop in "I enjoyed my feed" and 60% of the extra time comes from content types with the lowest enjoyment. Then "valued time" (time weighted by a satisfaction signal) may be flat or down even though raw time is up.

Options. (A) Ship, accepting the cost. (B) Change the objective: combine the engagement prediction with a prediction of "valued" (from surveys or "not interested" signals) and a penalty for low-quality content, then retest. (C) Ship only to segments where satisfaction holds. (D) Roll back.

Decision. B, with a six-week test and a long-term holdout, and a rule written in advance: launch if the survey score is no worse than -0.3 , retention's lower bound is above the margin and reports do not rise beyond +3%. Do not ship the +5% while the survey and the novelty read are open.

Creators and harm. A ranking that favors certain formats changes what creators make; check exposure for different creators, and escalate to policy and safety if the content type is harmful.

Company note. Meta is the setting only: Inferred from candidate reports of feed and engagement-versus-wellbeing prompts [unverified: candidate-reported]. No Meta data or practice is claimed.

Follow-ups.

- **Q.** How can survey results be trusted if only a few respond? **A.** "Treat them as a signal with an interval and bias in mind: sample randomly within the test, ask the same short question to both arms, check response rates by arm and by segment, and corroborate with behavior (not-interested taps, hiding posts, returning less). A drop that shows in several independent measures is credible."
- **Q.** What if leadership wants the time-spent gain for this quarter? **A.** "Show the cost in their terms: the novelty decay, the retention risk and the share of the gain that is unvalued time. Offer the version that keeps most of the gain with the guardrails satisfied (a changed objective), and a date for the decision. If they still decide to ship, ship to a limited share and keep the holdout so that the effect is measured."

IQ-C23-07 · Hard · ai · AI · Amazon

Prompt. How would you improve recommendations for new products with no history, when paid placements share the page?

Rubric.

- **Baseline:** "Recommend popular products."
- **Strong:** Uses content-based signals and exploration for cold start.
- **Standout:** Sizes the exploration budget, separates organic from sponsored placement, designs evaluation and guardrails, addresses seller fairness and trust, and avoids feedback loops.

Model answer. The setting is general; nothing here describes any real system. A recommender trained on purchases and clicks rarely shows new products, because they have no data; so they never get data.

Use what a new product does have. Its attributes (category, brand, price, images, description), the seller's history, and similarity to products shoppers already like. These content signals let the ranking guess before behavior exists.

Add an exploration budget. Show each new product to enough shoppers to estimate its click rate: about 1,000 impressions give a 95% interval of about ± 1.35 points at a 5% rate. With 5,000 new products a day that is 5,000,000 impressions, which is 0.25% of 2 billion daily impressions. If new products click at 3% against 6% for established ones, the cost is about 150,000 clicks a day, 0.125% of all clicks. Cheap relative to a catalog that never changes. Spread the exposure across shoppers and slots, stop showing an item once the estimate is clear, and weight toward items with strong content signals.

Sponsored placements. Paid placements mean a seller can buy exposure. Keep organic recommendations and paid placements distinct and clearly labeled, and make sure that the organic model is not trained on clicks that were driven by paid position (position bias and exposure bias). Otherwise paid products will look better than they are and the system will learn to favor them.

Evaluation. Online test with a holdout; measure purchases and revenue per session, return rates and satisfaction, new-product success (the share that reach a minimum sales level), and seller diversity. Guardrails: complaints, returns, and the share of the page that is paid.

Fairness and trust. New sellers need a chance; shoppers need to trust that recommendations are not only the highest bidder's. Publish clear rules and monitor concentration.

Company note. Amazon is the setting only: Inferred from candidate reports of recommendation and marketplace prompts [unverified: candidate-reported]. No Amazon data or practice is claimed.

Follow-ups.

- **Q.** How do you keep the exploration budget from becoming an advertising loophole? **A.** "Keep it algorithmic and separate from paid placement: eligibility by content quality signals, a cap per seller, and no payment influence. Audit who receives exploration exposure, so that the budget serves learning and not whoever pays."
- **Q.** What if new products have high return rates? **A.** "Treat returns as a guardrail and a label: reduce exposure for items with high returns after a minimum number of sales, and

use early return signals in ranking, since a click or a sale that is returned is not a success."

IQ-C23-08 · Hard · ai · AI · Microsoft

Prompt. Design the fairness checks for a system that recommends jobs to people.

Rubric.

- **Baseline:** "Remove gender and race from the data."
- **Strong:** Measures outcomes by group and considers proxies.
- **Standout:** Defines exposure and selection measures, tests significance, considers feedback loops and the difference between measuring and deciding, involves legal and policy, and sets monitoring and remedies.

Model answer. The setting is general; nothing here describes any real system. Removing sensitive attributes from the data does not remove bias, because other features (location, schools, past job titles, even names or the time of activity) can stand in for them, and because the data itself reflects past outcomes. So the check must measure outcomes.

What to measure. (1) *Exposure:* how often qualified people in each group are shown a given type of job, especially higher-paying ones. (2) *Outcomes:* application and response rates by group, for equally qualified users. (3) *Error rates:* whether the model's mistakes (qualified people not shown a job) are larger for some groups. Group definitions need care and legal guidance, and in many places the system may not hold the sensitive attribute, so audits may use opt-in data or a trusted third party.

An example check. Suppose among 10,000 equally qualified users in each of two groups, a job type is shown to 30% of group A and 21% of group B. The ratio of rates is $0.21 \div 0.30 = 0.70$. A commonly used rule of thumb in United States employment settings treats a ratio below 0.8 as a sign of possible adverse impact [V: Uniform Guidelines on Employee Selection Procedures (1978), 29 CFR 1607: the four-fifths rule is a rule of thumb and not determinative]. The standard error of the difference is about 0.6 points, so the 9-point gap is about 15 standard errors: not chance. That triggers an investigation, not an automatic verdict.

Find the cause. Is it the candidate stage (the job never enters the candidate set for group B), the ranking (features that proxy for group), the data (past hires), or the interface? Check the feedback loop: if users click what they are shown and the model learns from clicks, an early gap can grow.

Remedies. Re-examine features that act as proxies; balance exposure by adding candidates or re-ranking to meet an agreed floor; improve labels (use qualification signals, not past hiring alone); give users controls and transparency about why a job is shown.

Governance. Fairness definitions can conflict (equal exposure against equal accuracy), and choosing among them is a policy decision with legal, ethical and business input, not an engineering default. Document the choice, the tests, the thresholds for action, and the monitoring cadence.

Company note. Microsoft is the setting only: **Inferred** from candidate reports of recommendation and responsible-AI prompts [unverified: candidate-reported] . No Microsoft data or practice is claimed; the 0.8 ratio is a widely cited rule of thumb and the law varies by jurisdiction.

Follow-ups.

- **Q.** If you cannot collect the sensitive attribute, how do you check? **A.** "Use a voluntary, consented survey of a sample to get group labels for auditing only, use a trusted third party, or test with synthetic matched profiles that differ only in a proxy (a name or a school). Each has limits, which should be stated; none replaces monitoring outcomes."
- **Q.** What if equalizing exposure lowers relevance? **A.** "Measure the cost. Often the loss is small; if it is large, ask whether the model's relevance estimate itself reflects the bias. The tradeoff is a decision for the company with policy and legal input, with the numbers shown."

IQ-C23-09 · Hard · ai · AI · Stripe

Prompt. Design the decisioning for a payments fraud score: what happens at each score range?

Rubric.

- **Baseline:** "Block payments with a high fraud score."
- **Strong:** Uses zones (allow, review, block) with costs.
- **Standout:** Computes cost by zone, finds the review break-even, measures false declines, considers delayed labels, drift and feedback, and sets monitoring and rules around the model.

Model answer. The setting is general; nothing here describes any real system. A single block-or-allow threshold wastes the middle: payments that are risky but not clearly fraud. I'd use three zones.

Numbers (invented). 10,000,000 payments a day, 0.5% fraudulent: 50,000. Fraud costs \$95 per fraudulent payment (the loss plus fees). A human review costs \$1 and catches 80% of the fraud in the review zone. Blocking a legitimate payment costs \$5 of margin plus some customer goodwill.

Zones. **Block** (very high scores): 20,000 payments, 18,000 fraudulent, so 90% precision; 2,000 good payments are blocked. **Review** (middle): 150,000 payments with 25,000 fraudulent (16.7%); reviewers catch 20,000 and miss 5,000. **Allow** (low scores): 9,830,000 payments with 7,000 fraudulent.

Cost. Fraud that gets through: $(7,000 + 5,000) \times \$95 = \$1,140,000$. Review: $150,000 \times \$1 = \$150,000$. Blocked good payments: $2,000 \times \$5 = \$10,000$. Total about \$1.30 million a day, against \$4.75 million with no screening ($50,000 \times \$95$).

Where to put the review boundary. Moving payments into review pays if the fraud rate in the added band exceeds the break-even: cost of review per payment $\div$ (catch rate $\times$ fraud cost) $= \$1 \div (0.8 \times \$95) = 1.3\%$. If a band of 100,000 payments just below the review zone has a

3% fraud rate (3,000 frauds), reviewing it costs \$100,000 and prevents $0.8 \times 3,000 \times \$95 = \$228,000$: a net gain of \$128,000 a day. So extend review down to the point where the fraud rate falls toward 1.3%.

Beyond the score. Rules for known patterns (a card tested with many tiny charges), step-up verification for risky payments (ask for an extra check before blocking), and different thresholds by merchant type and amount. Measure the *false decline* rate and its effect on good customers and merchants, not just fraud caught.

Data realities. Labels arrive late (disputes come weeks later) and only for payments that were allowed; payments that were blocked never reveal whether they were fraud. That selection bias means the model's training data excludes what it blocked; keep a small random allowance or use review outcomes to learn. Fraudsters adapt, so the model drifts: monitor score distributions and fraud rates, retrain often, and keep a human-in-the-loop for new patterns.

Company note. Stripe is the setting only: **Public** that Stripe offers machine-learning-based fraud screening, **Inferred** from candidate reports of risk-scoring prompts [V: Stripe Radar machine-learning fraud screening; the prompt descriptions are candidate-reported and unverified]. No claim is made about any real system's numbers or design.

Follow-ups.

- **Q.** How do you learn about fraud in payments you blocked? **A.** "You mostly cannot, so the blocked set is a blind spot. Allow a small random sample of high-score payments through under strict limits to measure true fraud, use review outcomes, and compare against chargebacks from similar allowed payments. The sample costs a small amount of fraud and keeps the model honest."
- **Q.** A merchant complains that too many good customers are declined. What do you do? **A.** "Look at that merchant's decline rate and its review outcomes: if the false-decline rate is high, adjust its threshold or add step-up verification instead of blocking, share the data and the rules at a level that does not help fraudsters, and agree a target with the merchant, since its cost of a false decline differs from the average."

IQ-C23-10 · Hard · ai · AI

Prompt. A new ranking shows gains in a week-long test. How do you decide whether to launch?

Rubric.

- **Baseline:** "Launch; the test was positive."
- **Strong:** Checks guardrails, significance and novelty.
- **Standout:** Reads effects by week, plans a longer test and a long-term holdout, handles interference and the creator side, uses a pre-set rule, and states what an offline gain does and does not show.

Model answer. A one-week gain is the beginning of a decision, not the end. I'd ask five things.

Is it real? Check validity (sample ratio, logging) and that the result clears the pre-set threshold with an interval, not just a point estimate (C14).

Is it settling? Read the effect by day or week. Suppose the lift is +4.5% in week one, +3.1% in week two, +2.2% in week three and +1.8% in week four: each week is about 70% to 80% of the one before, so the effect is still falling and the week-one figure is mostly novelty. I would wait until it flattens, typically several weeks, before estimating the long-run effect.

What does it cost? Guardrails: retention, satisfaction surveys, reports, latency and cost. A gain in clicks with a fall in retention is a loss.

Does the test measure the whole system? Randomizing users may miss interference: in a feed, one user's content is seen by others; in a marketplace, one buyer's purchase removes supply. A ranking change also changes what creators do over weeks, which a short test cannot see. Where interference is likely, use cluster randomization or measure the other side of the market, and look at creator-side measures.

What is the long-term effect? Launch to most users while keeping a long-term holdout of 1% to 2% for several months to track retention and creator behavior, so the decision can be reversed.

Offline results. An offline improvement in predicting past behavior is a reason to test, not evidence of a product gain, because it reflects the old system's biases and cannot show how people react (the Netflix Prize lesson, CS-C23-02).

Decision. Write the rule first: primary threshold, guardrail margins, minimum duration and what to do in a mixed case. If the rule is met after the effect has settled, launch in stages with the holdout; if not, iterate.

Follow-ups.

- **Q.** How long should the test run? **A.** "Long enough for the effect to stop decaying and to cover a full weekly cycle and, for retention or creator effects, several weeks; practically four to eight weeks for ranking changes, plus a longer holdout after launch. A shorter test is acceptable only for low-risk, reversible changes."
- **Q.** How do you measure the effect on creators? **A.** "Track creator-side outcomes in the test: exposure and engagement by creator group, new-creator success, and posting behavior over time. If creators can be randomized or clustered separately from viewers, test that directly; otherwise compare exposure distributions and watch for changes in what is produced."

12. Exercises

Ten exercises. Each has a model solution and an alternative. Figures are assumptions unless marked, and the arithmetic was computed.

EX-C23-01 · Write a prediction brief

Problem. Fill in a prediction brief (FW-C23-01) for a model that predicts which deliveries will be late.

Context. A logistics product wants to warn customers early.

Data. None; reason.

Constraints. Six minutes.

Expected approach. FW-C23-01.

Solution. *Decision and actor:* warn the customer and, for high-risk deliveries, rebook the route or add capacity; the actors are customers and operations. *Label:* delivered later than the promised window; produced when the delivery completes, so labels arrive after the prediction (a delay of hours to days) and are clean but late. *Data:* route, distance, weather, carrier, past lateness by route and time; gaps: new routes and rare events (storms). *Costs:* a false warning annoys the customer and trains them to ignore warnings; a missed late delivery causes a complaint and a refund. *Evaluation:* held-out recent weeks, by region and carrier, and calibration (do 70% risk scores mean 70% late?). *Monitoring and retraining:* weekly, since conditions change; alert when predicted risk and actual lateness diverge.

Alternative solution. Start with a rule (any delivery with a delayed pickup is flagged) and compare the model to it.

Tradeoffs. Warning too often reduces trust in warnings; set a threshold by the cost of a false warning.

EX-C23-02 · Choose a threshold within capacity

Problem. A spam classifier on 2,000,000 messages a day (2% spam: 40,000) has two thresholds. Threshold X: recall 90%, precision 60%. Threshold Y: recall 97%, precision 25%. A wrongly flagged good message costs \$0.50 and a missed spam costs \$4. Reviewers can handle 70,000 flags a day. Compute flagged counts, false positives, false negatives and costs, and choose.

Context. Threshold choice.

Data. As above.

Constraints. Seven minutes.

Expected approach. FW-C23-02.

Solution. X: true positives 36,000; flagged $36,000 \div 0.6 = 60,000$; false positives 24,000; false negatives 4,000; cost $24,000 \times \$0.50 + 4,000 \times \$4 = \$12,000 + \$16,000 = \$28,000$. Y: true positives 38,800; flagged $38,800 \div 0.25 = 155,200$; false positives 116,400; false negatives 1,200; cost $116,400 \times \$0.50 + 1,200 \times \$4 = \$58,200 + \$4,800 = \$63,000$. X is cheaper (\$28,000 against \$63,000) and fits capacity ($60,000 \leq 70,000$); Y flags 155,200, more than twice capacity. Choose X. If wrongly flagged messages were automatically filtered with no review, the cost per false positive might be higher; revisit.

Alternative solution. Auto-filter the top-scored messages, send a middle band to review, and allow the rest.

Tradeoffs. Recall of 97% looks better; its false positives swamp the queue and annoy users.

EX-C23-03 · Base rate and precision

Problem. A classifier has 80% recall and a 1% false-positive rate. Compute its precision on 1,000,000 items when the base rate is 5%, 1% and 0.1%, and explain what changes.

Context. Why precision depends on the base rate.

Data. As above.

Constraints. Five minutes.

Expected approach. C23.3 and C12.

Solution. Base 5%: 50,000 bad, 950,000 good; true positives 40,000; false positives 9,500; precision $40,000 \div 49,500 = 80.8\%$. Base 1%: 10,000 bad; true positives 8,000; false positives 9,900; precision 44.7%. Base 0.1%: 1,000 bad; true positives 800; false positives 9,990; precision 7.4%. The classifier is identical in all three; only the proportion of bad items changed. When the target is rare, even a small false-positive rate produces many more false alarms than true catches. A threshold that is sensible at 5% can be far too loose at 0.1%.

Alternative solution. Show the same with a 0.1% false-positive rate: at a 0.1% base rate precision would be $800 \div (800 + 999) = 44.5\%$.

Tradeoffs. Reducing the false-positive rate is often the highest-value improvement for rare events.

EX-C23-04 · The cost of staging

Problem. A recommender serves 6,000 requests a second at peak. A ranking model takes 0.15 ms per item. The catalog has 12,000 eligible items per user. Compute the processor cores needed to score all eligible items and to score 600 candidates, and the ratio.

Context. Why candidate generation exists.

Data. As above.

Constraints. Five minutes.

Expected approach. Worked example 2.

Solution. All items: $12,000 \times 6,000 = 72$ million scores a second $\times 0.15$ ms = 10,800 processor-seconds a second: about 10,800 cores. Candidates: $600 \times 6,000 = 3.6$ million scores a second $\times 0.15$ ms = 540 cores. The ratio is 20 to 1. Candidate generation therefore saves 95% of the ranking compute, provided it does not drop good items (its quality is measured by recall).

Alternative solution. Use a lighter model for the first 3,000 and the heavy model on the top 300 (a cascade).

Tradeoffs. Fewer candidates cost less and risk missing good items; measure the recall of the candidate stage.

EX-C23-05 · Compare rankings with NDCG

Problem. Raters give the top five results grades 0 to 3. Ranking A returns [2, 3, 0, 1, 3]. The ideal ordering of the same results is [3, 3, 2, 1, 0], with DCG 6.32. Compute A's NDCG and say what it means.

Context. Search quality.

Data. $DCG = \text{sum of grade} \div \log_2(\text{position} + 1)$.

Constraints. Six minutes.

Expected approach. Worked example 4.

Solution. A's $DCG = 2 \div 1 + 3 \div 1.585 + 0 \div 2 + 1 \div 2.322 + 3 \div 2.585 = 2 + 1.893 + 0 + 0.431 + 1.161 = 5.48$. $NDCG = 5.48 \div 6.32 = 0.87$. The ranking has all the right results but puts a perfect result (3) at position 5 and a weaker one (2) at position 1; the position discount penalizes that. Precision at 5 (grades 2 and above counted relevant) would show 3 of 5 for both A and the ideal, hiding the difference.

Alternative solution. Compare mean NDCG across 300 queries with a standard error to see whether a difference is real.

Tradeoffs. NDCG depends on rater guidelines and the choice of how many positions to count.

EX-C23-06 · Correct for position bias

Problem. In a randomized slice, an item moved from position 1 to position 3 saw its click rate fall from 10% to 4%. Examination of positions suggests the probability that a user looks at position 2 is 0.6 and at position 3 is 0.4 relative to position 1. Item P is shown at position 3 with a 3% click rate; item Q is shown at position 1 with an 8% click rate. Which is better, after correcting for position?

Context. Learning from clicks.

Data. As above.

Constraints. Five minutes.

Expected approach. C23.6.

Solution. Correct each click rate by dividing by the relative chance of being looked at: P: $3\% \div 0.4 = 7.5\%$ equivalent at position 1; Q: $8\% \div 1 = 8\%$. Q is slightly better than P (8% against 7.5%), a much smaller gap than the raw 3% against 8%. The raw numbers would have led a model to rank P far below Q. The randomized slice gave the factor ($4\% \div 10\% = 0.4$ for position 3), consistent with the examination estimate. Correcting for position stops the model from learning that top positions are good.

Alternative solution. Randomize positions for a small share of traffic and compare items at the same position.

Tradeoffs. Randomization costs a little engagement; it gives unbiased estimates.

EX-C23-07 · Read a ranking experiment

Problem. A new recommendation ranking has: plays +2.4% (interval +1.9% to +2.9%); 28-day retention +0.1 points (−0.3 to +0.5), margin −0.3; diversity of titles watched −6% (−8% to −4%), margin −3%; weekly plays lift +3.5%, +2.8%, +2.2%, +1.9%. Decide.

Context. A launch decision.

Data. As above.

Constraints. Six minutes.

Expected approach. FW-C23-05 and FW-C17-04.

Solution. Validity assumed. Plays are up, but the lift is falling each week (3.5, 2.8, 2.2, 1.9), so the 2.4% overall overstates the settled effect. Retention's interval is within its margin (lower bound −0.3 is at the margin) and centered above zero: acceptable. Diversity is down 6% with an interval entirely beyond its margin of −3%: a breach. The ranking is concentrating plays on fewer titles. Decision: do not launch as is. Investigate which titles and segments concentrate, adjust re-ranking to protect variety (for example, a minimum share for less popular titles), and retest for four to six weeks with a holdout. If diversity is a stated product value, a breach is not offset by plays.

Alternative solution. Launch to the segments where diversity does not fall, if they can be identified.

Tradeoffs. Protecting variety may reduce plays slightly; decide how much plays are worth against catalog health.

EX-C23-08 · Size an exploration budget

Problem. A store adds 2,000 new items a day and serves 500,000,000 recommendation impressions a day. To estimate a new item's click rate (about 5%) to within ± 1.5 points you need $n = 1.96^2 \times 0.05 \times 0.95 \div 0.015^2$ impressions. Compute n , the daily budget, and its share of impressions.

Context. Exploration planning.

Data. As above.

Constraints. Five minutes.

Expected approach. Worked example 5.

Solution. $n = 3.84 \times 0.0475 \div 0.000225 =$ about 811 impressions per item. Budget: $2,000 \times 811 = 1,622,000$ impressions a day, which is $1,622,000 \div 500,000,000 = 0.32\%$ of impressions. If the new items click at 3% against 6% for established ones, the lost clicks are $1,622,000 \times 0.03 = 48,700$ a day, which is $48,700 \div (500,000,000 \times 0.06) = 0.16\%$ of all clicks. Exploration of every new item costs about a sixth of one percent of clicks.

Alternative solution. Allow a looser margin (± 3 points) for low-value items: n falls by about 75%, to about 203.

Tradeoffs. A tighter estimate costs more impressions; choose the margin by what decision it supports.

EX-C23-09 · A fairness check

Problem. A recommendation system shows a type of opportunity to 30% of group A and 27% of group B among equally qualified users (10,000 each). Compute the ratio, the standard error of the difference and whether it is distinguishable from chance. What next?

Context. Fairness audit.

Data. As above.

Constraints. Six minutes.

Expected approach. IQ-C23-08.

Solution. Ratio $0.27 \div 0.30 = 0.90$, above the 0.8 rule of thumb [V: Uniform Guidelines on Employee Selection Procedures (1978), 29 CFR 1607: the four-fifths rule is a rule of thumb and not determinative]. Standard error of the difference: $\sqrt{(0.30 \times 0.70 \div 10,000 + 0.27 \times 0.73 \div 10,000)} = \sqrt{(0.000021 + 0.0000197)} = 0.0064$; the 3-point gap is about 4.7 standard errors: not chance. So the gap is real even though it passes the 0.8 rule of thumb; the rule is a screen, not a standard of fairness. Next: find where it arises (candidates, ranking, data), consider whether a 3-point gap matters for this type of opportunity (it may for high-stakes ones), check whether it grows with the feedback loop, and decide with policy and legal input whether to act and how.

Alternative solution. Track the gap over time; a gap that widens each month is more concerning than a stable one.

Tradeoffs. Large samples detect small gaps; judgment is needed on which gaps to act on.

EX-C23-10 · A timed full answer

Problem. Give a 30-minute answer to: "Improve the search on an e-commerce app for shoppers who type queries in a mix of English and a local language."

Context. Self-scored; record yourself. State assumptions aloud.

Data. None. Label what you assume.

Constraints. Use the 30-minute budget in section 4.

Expected approach. FW-C23-08 and FW-C23-04, scored against the six behaviors in section 1.

Solution. An outline of a strong answer. Goal: shoppers find what they want quickly in the language they type; success is searches that lead to a purchase or an add to cart, with fewer reformulations. Diagnose by stage and segment: segment queries by language and script (English, local script, local language in Latin letters, mixed); check understanding (spelling, transliteration, synonyms across languages), retrieval (are products indexed under both languages' terms?) and ranking (do signals trained on English clicks work for others?). Evidence: rated samples per segment (needs raters who know the language), zero-result

rate, reformulation rate and conversion by segment; the overall average will hide a segment problem. Options: add transliteration and cross-language synonyms in query understanding, index products with multilingual attributes, create rater guidelines and test sets per language, and fall back to category browsing and popular items where data is thin (cold start).

Metrics: conversion per search, reformulations, zero-result rate by segment, NDCG on rated sets; guardrails: latency and the quality of English queries. Experiment: offline gate on rated sets, then a multi-week online test by segment with a holdout, reading novelty. Risks: thin data in some languages, feedback loops that favor English, and mistranslation errors; mitigate with raters and monitoring. Decision: start with the segment with the largest traffic and the largest gap.

Alternative solution. Start with a query-understanding fix (transliteration), which is cheap and testable, before changing ranking.

Tradeoffs. Per-language models and raters cost more; data in smaller languages is scarce, so content-based methods carry more weight.

13. Practice protocol

Timed drills (three a week for three weeks).

- *Week 1: predictions and thresholds.* Use IQ-C23-01, IQ-C23-02, EX-C23-01, EX-C23-02 and EX-C23-03. Do the confusion-table and base-rate arithmetic by hand and state the cost-based choice aloud.
- *Week 2: ranking, search and metrics.* Use IQ-C23-03, IQ-C23-04, IQ-C23-05, EX-C23-04, EX-C23-05, EX-C23-06 and EX-C23-07. Draw the recommendation funnel, compute NDCG for a pair of rankings, and read an experiment table.
- *Week 3: loops, fairness, risk and experiments.* Use IQ-C23-06 to IQ-C23-10, EX-C23-08, EX-C23-09 and EX-C23-10. Run a full 30-minute answer with a partner who changes the base rate or reveals a novelty decay midway.

With a partner. Ask your partner to say "It's 99% accurate," "Clicks are up," and "The offline metric improved" at random. Practice answering with a base rate, a satisfaction measure and an online test.

Read a real system. Read a public engineering or research write-up on how a product you use ranks or recommends (a search, feed or video service) and extract its stages, signals and metrics in one page.

Feedback checklist (yes, partly, no).

1. Did I say what the model predicts, from what label, and whether it matches the goal?
2. Did I use the base rate, costs and capacity for thresholds?
3. Did I describe the system by stage and place my intervention?
4. Did I use a metric stack beyond clicks, with long-term and guardrails?

5. Did I name data problems (position bias, label noise, cold start)?
6. Did I address feedback loops, popularity bias and exploration?
7. Did I plan the experiment with novelty, interference and a holdout?
8. Did I distinguish offline gains from product gains?
9. Did I consider creators, groups and fairness where relevant?
10. Did I close with a decision and what would change it?

Answer journal. For each drill record the date, the prompt, the label and metric stack, the stage you chose, the main risk, your score out of ten and the step you rushed.

14. Implementation playbook

Apply the chapter on real work over the next three weeks. If you do not work on ranking, use any prediction your product makes (a suggestion, a score, a filter). Each deliverable doubles as an interview story.

Days 1 to 3. Pick one prediction or ranking in your product and write its prediction brief (FW-C23-01): the decision, the label (and how it is produced), the data and gaps, the cost of each error and how it is evaluated. Ask the owning engineer what is missing.

Week 1. Find the base rate and the confusion table for a threshold in use (or a rule). Compute precision, recall and cost at the current threshold and one alternative.

Week 1. Draw the stages of the system (FW-C23-03) and note where each stage's quality is measured, if at all.

Week 2. List the metrics for the system and classify them by layer (engagement, satisfaction, long-term, diversity, guardrail). Identify one missing layer and propose a measure.

Week 2. Check for a feedback loop (FW-C23-06): measure concentration (the share of exposure going to the top items) and exposure for new items.

Week 3. Review one recent experiment on this system for novelty (effects by week), guardrails and interference. Write what you would add to the plan.

Deliverables that become stories: the prediction brief (a time you clarified what a model was for), the threshold or metric analysis (a time you used costs and base rates), and the loop check (a time you found a system learning from itself). Keep the numbers real, and keep what you did separate from what the team did.

15. Key takeaways and further reading

Key takeaways

1. A model is a means to a product decision: state what it predicts, from what label and data, who acts on it and what each error costs; a proxy label (a click) is not the goal (satisfaction), and the gap is where systems go wrong.
2. Set thresholds by the cost of each error, the base rate and the review capacity, not by accuracy; for rare events precision collapses when the base rate falls, and zones (act, review, allow) beat a single cut.
3. Ranking systems work in stages (candidate generation, ranking, re-ranking, presentation) for cost reasons; diagnose which stage fails and by which segment, since an average hides a class problem.
4. Judge ranked systems with a metric stack (engagement, satisfaction, long-term, diversity and fairness, guardrails), read effects by week for novelty, and treat offline gains as a gate to an online test with a long-term holdout.
5. Systems learn from their own outputs: popularity bias, position bias and creator incentives are feedback loops that need measurement and an exploration budget, and fairness needs outcome measures by group, not just removal of sensitive fields.

Further reading

- Xavier Amatriain and Justin Basilico, "Netflix Recommendations: Beyond the 5 Stars (Part 1)," Netflix Tech Blog, 6 April 2012 (netflixtechblog.com/netflix-recommendations-beyond-the-5-stars-part-1-55838468f429). The primary source for CS-C23-02.
- Carlos A. Gomez-Urbe and Neil Hunt, "The Netflix Recommender System: Algorithms, Business Value, and Innovation," *ACM Transactions on Management Information Systems* 6(4), Article 13 (2015) [V: ACM Transactions on Management Information Systems 6(4), Article 13, December 2015, doi 10.1145/2843948].
- Greg Linden, Brent Smith and Jeremy York, "Amazon.com Recommendations: Item-to-Item Collaborative Filtering," *IEEE Internet Computing* (2003) [V: Linden, Smith and York, *IEEE Internet Computing* 7(1), 2003, pp. 76 to 80].
- Ron Kohavi, Diane Tang and Ya Xu, *Trustworthy Online Controlled Experiments* (2020), on experiments for ranking and the pitfalls of short tests [V: Cambridge University Press, 2 April 2020, ISBN 9781108724265; Twyman's law is chapter 3].
- Solon Barocas, Moritz Hardt and Arvind Narayanan, *Fairness and Machine Learning* (2023), for a rigorous treatment of group measures and their tradeoffs [V: MIT Press, 19 December 2023, ISBN 9780262048613; also free online at fairmlbook.org].
- Chip Huyen, *Designing Machine Learning Systems* (2022), for the engineering of production models, monitoring and drift [V: O'Reilly, June 2022, ISBN 9781098107963].
- Google's published Search Quality Rater Guidelines (officially the Search Quality Evaluator Guidelines, first made public in an abridged form in 2013 and in full in November 2015, and revised regularly), as an example of how human judgments of

relevance are specified [V: Google Search Quality Evaluator Guidelines; full public version of 19 November 2015 (about 160 pages), an abridged public version in 2013 per trade press; revised since, with a September 2025 edition per a trade report].